ME3160 — Power & Refrigeration Cycles

Project 1 Support Material

Surrogate-Assisted Optimization of a Methane-Fired Combined Cycle

1. What Are You Doing?

You are building a physics model of a combined-cycle plant (Brayton topping cycle + HRSG + Rankine bottoming cycle), then finding the setting of five design variables — rp, TIT, Ps, Ts, Pcond — that maximizes overall efficiency without violating any of the six constraints in §1.4. The physics itself is fully specified in the problem statement; this document focuses on the optimization/ML pipeline layered on top of it (Parts B and C).

2. Why Are You Doing It This Way?

Your physics model (Part A) is expensive: every call solves a full variable-Cp Brayton cycle, an HRSG pinch analysis, and a CoolProp-based Rankine cycle. A brute-force grid search over all 5 design variables makes that cost explode.

> Grid search: 20 points per variable, 5 variables → **20^5 = 3,200,000 evaluations.**
>
> This pipeline: Latin Hypercube Sampling → **300–500 evaluations total** — roughly **6,000× fewer**.
>
> *Illustration only (not required in your report): if one physics evaluation takes ~0.3 s, a full grid search is ~11 days of continuous compute. LHS + surrogates finishes the same job in under 3 minutes of expensive-model time.*

That's the whole point of Parts B and C: sample the expensive model a few hundred times, fit a cheap stand-in (surrogate) to those samples, search the cheap surrogate as much as you like, and only go back to the expensive model to double-check the final answer.

3. Pipeline at a Glance

Part A — Build and validate the physics model.

Variable-Cp, validated against the three hand-calculable checks in §1.7. Everything below treats this as a finished, trustworthy black box.

Part B — Turn the expensive model into a cheap one.

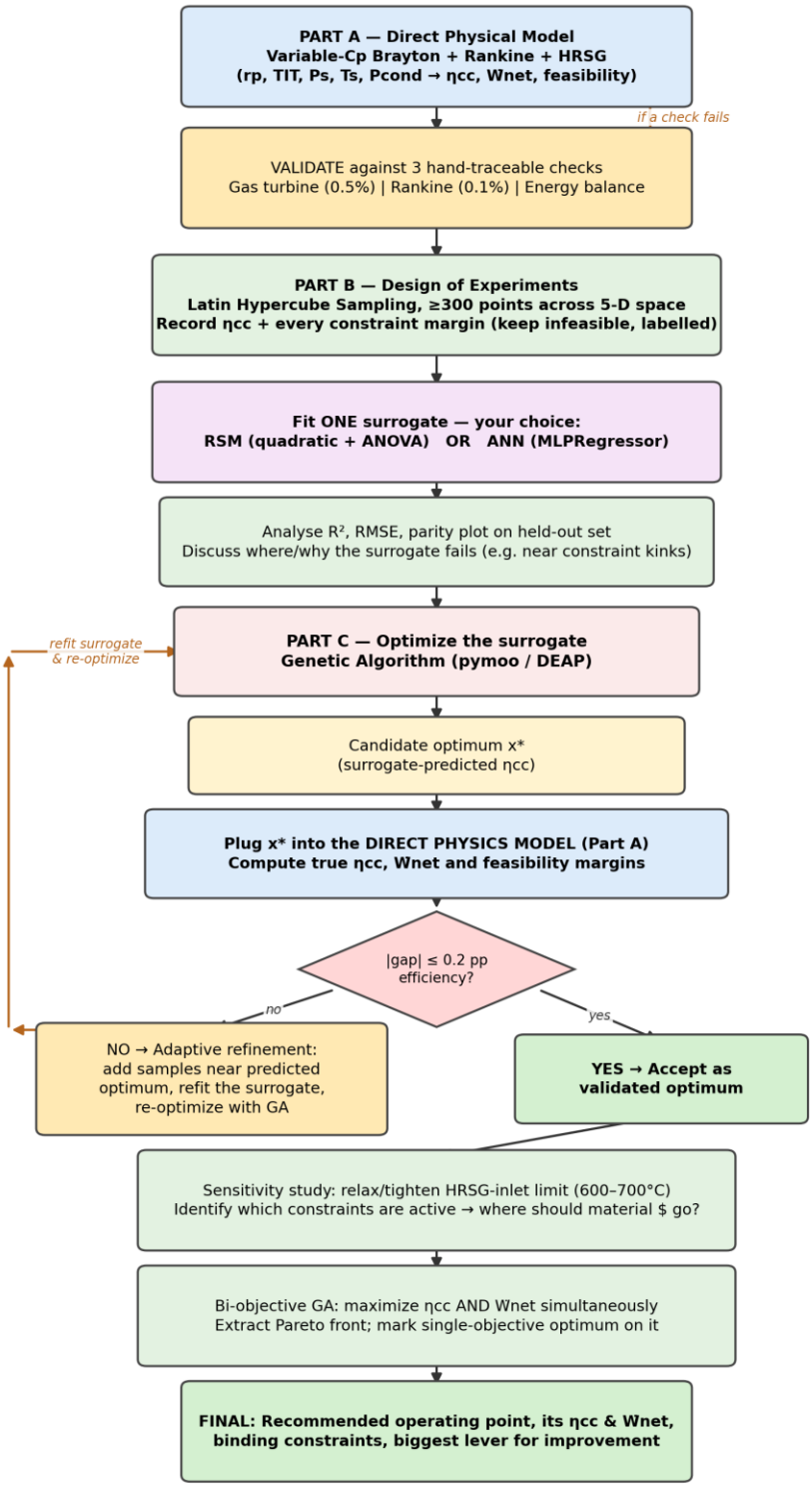
Sample the design space (LHS), then fit ONE surrogate — your choice of a quadratic Response Surface (RSM) or a small Neural Network (ANN), not both — and analyse it on held-out data.

Part C — Optimize the cheap model, then verify.

Optimize your surrogate with a Genetic Algorithm (pymoo/DEAP), re-check the winner against the real physics model, refine if it disagrees, run a sensitivity study, and produce a Pareto front (ηcc vs. net power) instead of a single answer.

4. Pipeline Flow Chart

Follow it top to bottom; the two loops (fail-validation, and adaptive refinement) are the only places you circle back.



5. Step-by-Step Pseudocode

This is structural pseudocode — it shows what goes into each stage and what comes out, not working thermodynamics. Every block assumes you already have the validated Part A model.

Physical Model — treated as a black box

You already built and validated this in Part A. Nothing below re-derives it — every later step just calls it.

> **`direct_model(rp, TIT, Ps, Ts, Pcond)  →  eta_cc, Wnet, margins, feasible`**
> *Internally: variable-Cp Brayton cycle → HRSG pinch analysis → CoolProp Rankine cycle. You know this part — it's Part A.*

Step 1 — Design of Experiments — Latin Hypercube Sampling (Part B)

INPUTS → *bounds for the 5 design variables; the direct_model() black box; number of samples N (≥300, 500 recommended).*

```
from scipy.stats import qmc

sampler = qmc.LatinHypercube(d=5, seed=SEED)
unit_samples = sampler.random(n=500)
X = qmc.scale(unit_samples, l_bounds, u_bounds)   # scale to real engineering units

records = []
for x in X:
    eta_cc, Wnet, margins, feasible = direct_model(*x)
    records.append({'x': x, 'eta_cc': eta_cc, 'Wnet': Wnet,
                    'margins': margins, 'feasible': feasible})
df = to_dataframe(records)   # KEEP infeasible rows -- label them, do not drop
```

OUTPUT → *df — one row per sampled design, with ηcc, Ẇnet, all 6 constraint margins, and a feasible flag (infeasible rows kept and labelled).*

Step 2 — Fit YOUR surrogate — choose RSM or ANN, not both

INPUTS → *df from Step 1 — the design-variable columns (X) and the eta_cc column (plus the binding constraint-margin columns).*

Implement ONE of these two options. Both are shown here only so you can pick whichever fits your group's background — the project asks for a single surrogate.

Option A — Response Surface (RSM) + ANOVA

```
import statsmodels.api as sm
from statsmodels.formula.api import ols

formula = 'eta_cc ~ (rp + TIT + Ps + Ts + Pcond)**2 + ' \
          'I(rp**2) + I(TIT**2) + I(Ps**2) + I(Ts**2) + I(Pcond**2)'
rsm_model = ols(formula, data=df_feasible).fit()
anova_table = sm.stats.anova_lm(rsm_model, typ=2)

rsm_margin_models = {c: ols(f'{c} ~ ...same formula...', data=df).fit()
                     for c in binding_constraints}
```

Option B — Neural Network (ANN)

```
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV

X_train, X_test, y_train, y_test = train_test_split(X, eta_cc, test_size=0.2,
random_state=SEED)
scaler = StandardScaler().fit(X_train)
Xs_train, Xs_test = scaler.transform(X_train), scaler.transform(X_test)

param_grid = {'hidden_layer_sizes': [(10,10), (20,10), (20,20)],
              'activation': ['tanh', 'relu'], 'alpha': [1e-4, 1e-3, 1e-2]}
search = GridSearchCV(MLPRegressor(max_iter=5000, random_state=SEED),
                      param_grid, cv=5, scoring='r2')
search.fit(Xs_train, y_train)
ann_model = search.best_estimator_
```

OUTPUT → *surrogate_model — your fitted RSM or ANN, callable to predict ηcc (and the binding constraint margins) at any x.*

Step 3 — Analyse your surrogate on held-out data

INPUTS → *surrogate_model from Step 2, and the held-out X_test / y_test (for ANN, your 20% split; for RSM, a held-out subset you set aside the same way).*

```
from sklearn.metrics import r2_score, mean_squared_error

y_pred = surrogate_predict(X_test)
r2   = r2_score(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False)
plot_parity(y_test, y_pred, title=f'R2={r2:.3f}, RMSE={rmse:.4f}')
# look specifically near the HRSG-inlet / pinch boundary -- a quadratic RSM
# should show visible bias there; discuss why, whichever surrogate you used
```

OUTPUT → *R^2, RMSE, and a parity plot — feeds directly into your report's surrogate-quality discussion (where and why it fails).*

Step 4 — Optimize the surrogate with a Genetic Algorithm (Part C)

INPUTS → *surrogate_model as a callable prediction function, plus the bounds and constraint-margin functions from Step 1.*

```
from pymoo.algorithms.soo.nonconvex.ga import GA
from pymoo.optimize import minimize as pymoo_minimize

problem = CombinedCycleProblem(surrogate_predict, surrogate_margin_fn, bounds)
res = pymoo_minimize(problem, GA(pop_size=100), ('n_gen', 200), seed=SEED)

best_candidate = CandidateResult(x=res.X, eta=-res.F[0])   # pymoo minimizes,
                                                            # so negate eta_cc
```

OUTPUT → *best_candidate — the GA's best feasible design (x*, surrogate-predicted ηcc).*

Step 5 — Direct-model verification & adaptive refinement

INPUTS → *best_candidate.x from Step 4, and the direct_model() black box.*

```
eta_direct, Wnet_direct, margins, feasible = direct_model(*best_candidate.x)
gap = best_candidate.eta - eta_direct

iteration = 0
while abs(gap) > 0.002 and iteration < MAX_REFINEMENTS:     # 0.2 percentage points
    new_points = latin_hypercube_around(best_candidate.x, radius=trust_region, n=50)
    new_records = [direct_model(*pt) for pt in new_points]
    df = df.append(new_records)
    refit_surrogate()      # refit your RSM or ANN on the augmented df
    best_candidate = re_optimize_with_ga()
    eta_direct, Wnet_direct, margins, feasible = direct_model(*best_candidate.x)
    gap = best_candidate.eta - eta_direct
    iteration += 1
```

OUTPUT → *final_design — a design whose surrogate-predicted ηcc has been confirmed against the real physics model to within 0.2 pp.*

Step 6 — Sensitivity study

INPUTS → *the full Steps 1–5 pipeline, re-run once per HRSG-inlet limit.*

```
for hrsg_limit_C in [600, 650, 700]:
    set_constraint_ii_limit(hrsg_limit_C)
    result = repeat_optimization_pipeline()
    record(hrsg_limit_C, result.x_star, result.eta_cc, result.active_constraints)

# which constraints are active at each limit? where would loosening a material
# limit buy the most efficiency? -> that's where materials R&D $ should go
```

OUTPUT → *a table of (limit, x*, ηcc, active constraints) used to argue where materials investment matters most.*

Step 7 — Bi-objective Pareto front (ηcc vs. net power)

INPUTS → *bounds, constraint-margin function, and ηcc / Ẇnet predictions (surrogate or direct — state which you use).*

```
from pymoo.algorithms.moo.nsga2 import NSGA2

class BiObjectiveProblem(Problem):
    def _evaluate(self, X, out):
        eta = np.array([eta_fn(x) for x in X])
        Wnet = np.array([wnet_fn(x) for x in X])
        out['F'] = np.column_stack([-eta, -Wnet])       # maximize both -> minimize
negatives
        out['G'] = np.array([constraint_margins_vec(x) for x in X]) * -1  # pymoo: G<=0
feasible

res = pymoo_minimize(BiObjectiveProblem(), NSGA2(pop_size=100), ('n_gen', 200), seed=SEED)
pareto_front = res.F, res.X
plot_pareto(pareto_front, mark_point=final_design)
```

OUTPUT → *pareto_front — the set of non-dominated designs to plot, with the single-objective optimum marked on it.*

6. Resources — Surrogate Modelling & Optimization

Videos build intuition; the documentation is what you actually code against.

Topic	YouTube video	Documentation
Latin Hypercube Sampling (DOE)	Random Sampling Tutorial 4 — Latin Hypercube Sampling youtube.com/watch?v=H0RZ1uezuuw	scipy.stats.qmc.LatinHypercube docs.scipy.org/doc/scipy/reference/generated/scipy.stats.qmc.LatinHypercube.html
Response Surface Methodology + ANOVA	RSM: Introduction to Response Surface Methodology youtube.com/watch?v=ERSWvYybOrk	statsmodels anova_lm statsmodels.org/devel/generated/statsmodels.stats.anova.anova_lm.html
Neural network intuition	But what is a neural network? — 3Blue1Brown youtube.com/watch?v=aircAruvnKk	scikit-learn MLPRegressor guide scikit-learn.org/stable/modules/neural_networks_supervised.html
Genetic Algorithms	The Knapsack Problem & Genetic Algorithms — Computerphile youtube.com/watch?v=MacVqujSXWE	pymoo documentation & tutorials pymoo.org
Pareto fronts / bi-objective optimization	Multiobjective optimization & the Pareto front youtube.com/watch?v=act0oZoV3RA	pymoo — Multi-Objective Optimization (NSGA-II) pymoo.org
Steam properties (Rankine side)	—	CoolProp High-Level Interface (PropsSI) coolprop.org/coolprop/HighLevelAPI.html

Note: RSM and GA tutorials online skew toward manufacturing/Minitab examples rather than Python. Use them for the concept, and use Section 5 to bridge to actual scipy/statsmodels/scikit-learn/pymoo code.